

```
1: Test Preview: Summary for kw1425 of c1
2: PPT 21
3: -----
4:
5:   Public Tests:
6:     Compiles:      1 / 1
7:     Tests Pass:    1 / 1
8:     Style Check:   1 / 1
9:
10: Git Repo: git@gitlab.doc.ic.ac.uk:lab2526_autumn/kotlinwebserver_kw1425.git
11: Commit ID: 36555
12: Git Diff: https://gitlab.doc.ic.ac.uk/lab2526_autumn/kotlinwebserver_kw1425/-/compare/master..36555?from_project_id=134045
```

```
1: package webserver
2:
3: // provided files
4:
5: class Request(
6:     val url: String,
7:     val authToken: String = ""
8: )
9:
10: class Response(
11:     val status: Status,
12:     val body: String = ""
13: )
14:
15: enum class Status(
16:     val code: Int
17: ) {
18:     OK(200),
19:     FORBIDDEN(403),
20:     NOT_FOUND(404)
21: }
```

```

1: package webserver
2:
3: import kotlin.text.substringBefore
4:
5: // write your web framework code here:
6:
7: fun scheme(url: String): String = url.substringBefore(':')
8:
9: fun host(url: String): String = url.substringAfter("://").substringBefore('/')
10:
11: fun path(url: String): String = url.substringBefore('?')
12:   .substringAfter(host(url))
13:
14: fun queryParams(url: String): List<Pair<String, String>> {
15:     val queryString = url.substringAfter('?')
16:     if (!url.contains('?')) {
17:         return emptyList()
18:     }
19:     val queries = queryString.split('&')
20:     .map { param ->
21:         val key = param.substringBefore('=')
22:         val value = param.substringAfter('=', "")
23:         Pair(key, value)
24:     }
25: }
26:
27: // http handlers for a particular website...
28:
29: fun homePageHandler(request: Request): Response = Response(Status.OK, "This is
Imperial.")
30:
31: fun homePageComputingHandler(request: Request): Response = Response(Status.OK,
"This is DoC.")
32:
33: fun helloHandler(request: Request): Response {
34:     val name = queryParams(request.url).filter { it.first == "name" }
35:     val style = queryParams(request.url).filter { it.first == "style" }
36:     if (name.any()) {
37:         val name = name[0].second
38:         if (style.any() && style[0].second == "shouting") return Response(Status.OK,
String.format("HELLO, %s!", name.uppercase()))
39:         return Response(Status.OK, String.format("Hello, %s!", name))
40:     } else {
41:         return Response(Status.OK, "Hello, World!")
42:     }
43: }
44:
45: fun errorHandler(request: Request): Response = Response(Status.NOT_FOUND)
46:
47: fun route(request: Request): Response {
48:     val route = path(request.url)
49:     when (route) {
50:         "/" -> return homePageHandler(request)
51:         "/computing" -> return homePageComputingHandler(request)
52:         "/say-hello" -> return helloHandler(request)
53:         else -> return errorHandler(request)
54:     }
55: }

```

✓ 2/2

✓

2/2

✓ 2/2

3/4

→ you could use split here instead

✓

13
20

{ can extract this to a helper

2 /4

→ you could simplify this by removing this branch and setting name to "World" when empty

2/2

Test Preview	WebServerTest.kt: 1/2	Katie Wan(kw1425):c1:21	Test Preview	WebServerTest.kt: 2/2	Katie Wan(kw1425):c1:21
	<pre>1: package webserver 2: 3: import org.junit.Test 4: import kotlin.test.assertEquals 5: 6: class WebServerTest { 7: @Test 8: fun `can extract scheme`() { 9: assertEquals("http", scheme("http://www.imperial.ac.uk/")) 10: assertEquals("https", scheme("https://www.imperial.ac.uk/")) 11: } 12: 13: @Test 14: fun `can extract host`() { 15: assertEquals("www.imperial.ac.uk", host("http://www.imperial.ac.uk/")) 16: assertEquals("www.imperial.ac.uk", host("https://www.imperial.ac.uk/")) 17: assertEquals("www.imperial.ac.uk", host("https://www.imperial.ac.uk/computing")) 18: } 19: 20: @Test 21: fun `can extract path`() { 22: assertEquals("/", path("http://www.imperial.ac.uk/")) 23: assertEquals("/", path("https://www.imperial.ac.uk/")) 24: assertEquals("/computing", path("https://www.imperial.ac.uk/computing")) 25: assertEquals("/computing/programming", path("https://www.imperial.ac.uk/computing/programming")) 26: assertEquals("/computing", path("https://www.imperial.ac.uk/computing?q=abc")) 27: } 28: 29: @Test 30: fun `can extract query params`() { 31: assertEquals(listOf(Pair("q", "xxx")), queryParams("http://www.imperial.ac.uk/?q=xxx")) 32: assertEquals(listOf(Pair("q", "xxx"), Pair("rr", "zzz")), queryParams("http://www.imperial.ac.uk/?q=xxx&rr=zzz")) 33: } 34: 35: @Test 36: fun `when no query params in url, empty list is extracted`() { 37: assertEquals(listOf(), queryParams("http://www.imperial.ac.uk/")) 38: } 39: 40: // ***** Tests for Handlers ***** 41: 42: @Test 43: fun `says hello world`() { 44: val request = Request("http://www.imperial.ac.uk/say-hello") 45: assertEquals("Hello, World!", helloHandler(request).body) 46: } 47: 48: @Test 49: fun `can be customised with particular name`() { 50: val request = Request("http://www.imperial.ac.uk/say-hello?name=Fred") 51: assertEquals("Hello, Fred!", helloHandler(request).body) 52: } 53: 54: @Test 55: fun `can process multiple params`() { 56: val request = Request("http://www.imperial.ac.uk/say-hello?name=Fred&style=shouting") 57: assertEquals("HELLO, FRED!", helloHandler(request).body) 58: } 59: 60: // ***** Tests for Routing *****</pre>			<pre>61: 62: @Test 63: fun `can route to hello handler`() { 64: val request = Request("http://www.imperial.ac.uk/say-hello?name=Fred") 65: assertEquals("Hello, Fred!", route(request).body) 66: } 67: 68: @Test 69: fun `can route to homepage handler`() { 70: assertEquals("This is Imperial.", route(Request("http://www.imperial.ac.uk/").body)) 71: assertEquals("This is DoC.", route(Request("http://www.imperial.ac.uk/computing").body)) 72: } 73: 74: @Test 75: fun `gives 404 when no matching route`() { 76: assertEquals(Status.NOT_FOUND, route(Request("http://www.imperial.ac.uk/not-here").status)) 77: } 78: 79: // ***** Tests for the Extensions ***** 80: 81: // *** More flexible routing *** 82: 83: // @Test 84: // fun `calling configureRoutes() returns app which can handle requests`() { 85: // 86: // val app = configureRoutes(...) 87: // 88: // assertEquals("This is Imperial.", route(Request("http://www.imperial.ac.uk/").body).body) 89: // assertEquals("This is DoC.", route(Request("http://www.imperial.ac.uk/computing").body).body) 90: // } 91: 92: // *** Filters *** 93: 94: // @Test 95: // fun `filter prevents access to protected resources`() { 96: // 97: // val app = configureRoutes(...) 98: // 99: // val request = Request("http://www.imperial.ac.uk/exam-marks") 100: // assertEquals(Status.FORBIDDEN, app(request).status) 101: // } 102: 103: // @Test 104: // fun `filter allows access to protected resources with token`() { 105: // 106: // val app = configureRoutes(...) 107: // 108: // val request = Request("http://www.imperial.ac.uk/exam-marks", "password1") 109: // assertEquals(Status.OK, app(request).status) 110: // assertEquals("This is very secret.", app(request).body) 111: // } 112: }</pre>	
git@gitlab.doc.ic.ac.uk:lab2526_autumn/kotlinwebserver_kw1425.git	36555	git@gitlab.doc.ic.ac.uk:lab2526_autumn/kotlinwebserver_kw1425.git	36555	git@gitlab.doc.ic.ac.uk:lab2526_autumn/kotlinwebserver_kw1425.git	36555

Test Preview

testResults.txt: 1/1

Katie Wan(kw1425):c1:21

```
1: ----- Test Output -----
2: Running LabTS build... (Fri 14 Nov 16:16:14 UTC 2025)
3:
4: Submission summary...
5: You made 2 commits
6:   - 2dd6349 finish [4 files changed, 82 insertions, 35 deletions]
7:   - 365554d fix [2 files changed, 14 insertions, 10 deletions]
8:
9: Preparing...
10:
11: BUILD SUCCESSFUL in 1s
12:
13: Compiling...
14: BUILD SUCCESSFUL in 11s
15:
16: Running tests...
17:
18: WebServerTest > says hello world PASSED
19:
20: WebServerTest > can extract scheme PASSED
21:
22: WebServerTest > can process multiple params PASSED
23:
24: WebServerTest > can route to hello handler PASSED
25:
26: WebServerTest > can be customised with particular name PASSED
27:
28: WebServerTest > can route to homepage handler PASSED
29:
30: WebServerTest > can extract host PASSED
31:
32: WebServerTest > can extract path PASSED
33:
34: WebServerTest > when no query params in url, empty list is extracted PASSED
35:
36: WebServerTest > can extract query params PASSED
37:
38: WebServerTest > gives 404 when no matching route PASSED
39:
40: BUILD SUCCESSFUL in 1s
41:
42: Checking code style...
43: BUILD SUCCESSFUL in 3s
44: Finished auto test. (Fri 14 Nov 16:17:04 UTC 2025)
45:
46: ----- Test Errors -----
47:
```